Théorie des graphes et optimisation

Informatique — Licence 2 (L2)

Cours INF-L2-16

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Résoudre des problèmes d'itinéraires et de planification grâce aux graphes : définitions, représentations, parcours en largeur et en profondeur, plus courts chemins de Dijkstra, arbres couvrants minimaux de Kruskal et Prim, coloration et applications au transport, aux réseaux et à la vie quotidienne.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Définitions : graphes orientés et non orientés
• Leçon 2 — Représentations : matrices et listes d'adjacence
• Leçon 3 — Parcours en largeur et parcours en profondeur
• Leçon 4 — Chemins les plus courts : l'algorithme de Dijkstra
• Leçon 5 — Arbres couvrants minimaux : Kruskal et Prim
• Leçon 6 — Coloration de graphes et problèmes de planification
• Leçon 7 — Applications : réseaux, transport et réseaux sociaux
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Un plan de ville, un réseau d'amis, les liaisons entre ordinateurs, les vols entre aéroports : derrière ces situations très différentes se cache la même structure mathématique, le graphe. Un graphe ne retient que l'essentiel — des points et des liaisons entre ces points — et cette simplicité est sa force : une fois un problème traduit en graphe, des algorithmes puissants et généraux prennent le relais. C'est ainsi que ton GPS choisit un itinéraire, qu'un opérateur raccorde des quartiers à la fibre optique au moindre coût et qu'une université construit un planning d'examens sans conflit. La théorie des graphes est née en 1736, quand Leonhard Euler a résolu le problème des ponts de Königsberg en le réduisant à des points et des traits : presque trois siècles plus tard, elle est devenue l'un des outils les plus concrets de l'informatique moderne, de la recherche opérationnelle à l'intelligence artificielle.

Ce cours prolonge la leçon d'initiation aux graphes du cours INF-L2-11 (Structures de données avancées), où nous avions découvert les parcours. Nous allons ici plus loin : représentations efficaces, plus courts chemins pondérés avec l'algorithme de Dijkstra, arbres couvrants minimaux avec Kruskal et Prim, et coloration de graphes appliquée à la planification. Chaque algorithme est d'abord compris à la main, sur de petits exemples entièrement calculés, avant d'être relié à ses applications réelles. La démarche est résolument pratique : aucun algorithme n'est seulement énoncé, il est exécuté entièrement sur un exemple chiffré, avec ses étapes intermédiaires, ses choix et ses pièges classiques. Tu verras ainsi pourquoi Dijkstra refuse les poids négatifs, pourquoi l'ordre des sommets change le résultat d'une coloration gloutonne, et

pourquoi deux algorithmes aussi différents que Kruskal et Prim aboutissent toujours au même coût minimal. Cette rigueur du calcul à la main est la meilleure préparation aux travaux pratiques, où les mêmes algorithmes seront programmés en Python.

Le fil rouge du cours est un graphe unique : six quartiers de Bamako (ACI 2000, Hippodrome, Médina Coura, Badalabougou, Lafiabougou, Magnambougou) reliés par dix routes dont on connaît les distances en kilomètres. Nous le représenterons, le parcourrons, nous y chercherons des itinéraires optimaux, puis nous y brancherons un réseau de fibre optique au moindre coût. À la fin du cours, tu sauras reconnaître, face à un problème nouveau, s'il relève d'un parcours, d'un plus court chemin, d'un arbre couvrant ou d'une coloration — c'est là le vrai savoir-faire de la théorie des graphes. Le cours suppose connus les rudiments de programmation Python vus en première année et la leçon d'initiation aux graphes du cours INF-L2-11 ; chaque leçon se termine par un exercice corrigé en détail, et les travaux dirigés, travaux pratiques et QCM finaux permettent de vérifier l'ensemble avant l'examen.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Définir un graphe par ses sommets et ses arêtes, et distinguer graphes orientés et graphes non orientés.
- Calculer les degrés des sommets et vérifier le lemme des poignées de main.
- Représenter un graphe par une matrice d'adjacence et par des listes d'adjacence, puis choisir la représentation adaptée à un problème.
- Exécuter à la main un parcours en largeur (BFS) et un parcours en profondeur (DFS) en suivant l'évolution de la file ou de la pile.
- Appliquer l'algorithme de Dijkstra pour obtenir les plus courts chemins depuis un sommet, et expliquer pourquoi il exige des poids positifs.
- Construire un arbre couvrant minimal avec les algorithmes de Kruskal et de Prim.
- Colorier un graphe par l'algorithme glouton et mobiliser la coloration pour résoudre un problème de planification (examens, emplois du temps).

Leçon 1 — Définitions : graphes orientés et non orientés

Un graphe est la donnée de deux ensembles : un ensemble S de sommets (les points) et un ensemble A d'arêtes (les liaisons entre points). On note le graphe G = (S, A) et on appelle ordre du graphe le nombre de ses sommets. Deux sommets reliés par une arête sont dits adjacents, ou voisins ; l'arête est alors incidente aux deux sommets. Cette définition volontairement pauvre est universelle : un sommet peut être un quartier, une page web, une personne ou un état d'un programme, et une arête n'importe quelle relation entre deux de ces objets — une route, un lien hypertexte, une amitié, une transition.

Dans un graphe non orienté, une arête relie deux sommets sans privilégier de sens : si la route relie le quartier X au quartier Y, on peut la parcourir dans les deux directions. Dans un graphe orienté, chaque liaison est un arc, fléché d'un sommet de départ vers

un sommet d'arrivée : une rue à sens unique, un lien d'une page web vers une autre, ou un transfert d'argent sont des arcs. Le degré d'un sommet, dans le cas non orienté, est le nombre d'arêtes qui le touchent. Dans le cas orienté, on distingue le degré entrant (nombre d'arcs qui arrivent) et le degré sortant (nombre d'arcs qui partent). Une arête qui relie un sommet à lui-même est une boucle ; un graphe sans boucle et sans arête double est dit simple.

Un graphe est pondéré (ou valué) lorsque chaque arête porte un nombre, son poids : une distance en kilomètres, un temps de trajet, un coût en francs. Un chemin est une suite de sommets où chacun est adjacent au suivant ; sa longueur est le nombre d'arêtes utilisées, ou la somme des poids si le graphe est pondéré. Un cycle est un chemin qui revient à son point de départ. Le graphe est connexe si tout sommet est atteignable depuis tout autre par un chemin : un graphe de routes où un quartier serait isolé ne serait pas connexe, et plusieurs problèmes du cours n'auraient aucun sens. Enfin, un résultat à connaître par cœur : dans tout graphe non orienté, la somme des degrés vaut le double du nombre d'arêtes, car chaque arête compte pour deux extrémités. C'est le lemme des poignées de main — si chacun serre la main d'un autre, chaque poignée de main fait intervenir exactement deux mains.

L'essentiel
Un graphe G = (S, A) relie des sommets par des arêtes (non orientées) ou des arcs (orientés). Degré = nombre d'arêtes touchant un sommet ; somme des degrés = 2 × nombre d'arêtes. Un graphe peut être pondéré (poids sur les arêtes), et il est connexe si tous ses sommets se rejoignent par des chemins.

Exemple — Le fil rouge du cours : six quartiers de Bamako

```
LE FIL ROUGE : SIX QUARTIERS DE BAMAKO

Sommets (les quartiers) :
ACI = ACI 2000 HIP = Hippodrome
MED = Medina Coura BAD = Badalabougou
LAF = Lafiabougou MAG = Magnambougou

Aretes (routes directes, poids = distance en km) :
ACI-HIP : 2 ACI-MED : 4 ACI-BAD : 3
HIP-MED : 2 HIP-LAF : 6
MED-LAF : 3 MED-MAG : 6
BAD-MAG : 2 BAD-LAF : 7 LAF-MAG : 4

Degres (nombre de routes directes) :
deg(ACI) = 3 deg(HIP) = 3 deg(MED) = 4
deg(BAD) = 3 deg(LAF) = 4 deg(MAG) = 3
Somme des degres = 20 = 2 x 10 aretes
(lemme des poignees de main)

Le graphe est non oriente (routes a double sens),
pondere (chaque arete porte un poids en km) et connexe
(tous les quartiers sont atteignables depuis ACI).
```

À retenir

• G = (S, A) : sommets + arêtes. Orienté = arcs fléchés (degré entrant / degré sortant) ; non orienté = arêtes utilisables dans les deux sens.

• Lemme des poignées de main : somme des degrés = 2 × nombre d'arêtes — un contrôle rapide pour vérifier un graphe dessiné ou saisi.

• Pondéré, connexe, chemin, cycle, boucle : un petit vocabulaire précis qui conditionne le choix de l'algorithme à appliquer.

Exercice

Une entreprise de livraison modélise ses tournées par un graphe orienté de sommets A, B, C, D et d'arcs : A→B, A→C, B→C, C→A, C→D, D→B (les rues à sens unique du centre-ville). 1) Pour chaque sommet, donne le degré sortant et le degré entrant. 2) Vérifie que la somme des degrés sortants égale le nombre d'arcs. 3) Ce graphe est-il simple ? Existe-t-il un chemin orienté de A vers D ?

✔ Corrigé

1) Degrés sortants : A envoie vers B et C, soit d+(A) = 2 ; d+(B) = 1 (vers C) ; d+(C) = 2 (vers A et D) ; d+(D) = 1 (vers B). Degrés entrants : d-(A) = 1 (depuis C) ; d-(B) = 2 (depuis A et D) ; d-(C) = 2 (depuis A et B) ; d-(D) = 1 (depuis C). 2) Somme des degrés sortants : 2 + 1 + 2 + 1 = 6, et la somme des degrés entrants vaut aussi 6 : c'est exactement le nombre d'arcs, car chaque arc a un seul départ et une seule arrivée — l'équivalent orienté du lemme des poignées de main. 3) Le graphe est simple : aucune boucle, aucun arc répété. Un chemin orienté de A vers D existe : A→C→D, qui respecte le sens de chaque arc.

Leçon 2 — Représentations : matrices et listes d'adjacence

Pour faire travailler un ordinateur sur un graphe, il faut le représenter en mémoire. La première méthode est la matrice d'adjacence : un tableau carré de taille n × n (n = nombre de sommets) où la case (i, j) vaut 1 si une arête relie le sommet i au sommet j, et 0 sinon. Pour un graphe pondéré, on y inscrit le poids de l'arête, et une valeur spéciale — 0 ou l'infini selon les conventions — quand il n'y a pas d'arête. La matrice a deux grands avantages : tester si deux sommets sont voisins coûte une seule lecture de case, et pour un graphe non orienté elle est symétrique par rapport à la diagonale, ce qui offre un moyen de contrôle visuel immédiat. Son défaut est sa gourmandise : elle occupe n^2 cases même si le graphe contient très peu d'arêtes.

La seconde méthode est la liste d'adjacence : pour chaque sommet, on conserve la liste de ses voisins (avec les poids, si le graphe est pondéré). La mémoire occupée est alors proportionnelle au nombre de sommets plus deux fois le nombre d'arêtes — chaque arête apparaît dans la liste de ses deux extrémités. Pour les graphes creux, c'est-à-dire ceux qui ont peu d'arêtes par rapport au maximum possible (réseaux routiers, réseaux sociaux, web), la liste est donc bien plus économe que la matrice. En contrepartie, savoir si deux sommets précis sont voisins oblige à parcourir la liste de l'un d'eux, et énumérer les voisins d'un sommet est en revanche immédiat — c'est exactement ce que font les algorithmes de parcours et Dijkstra.

Comment choisir ? Si le graphe est dense (presque toutes les paires de sommets sont reliées) ou si le programme teste sans arrêt des voisinages, la matrice s'impose. Si le graphe est creux — cas le plus fréquent en pratique — la liste d'adjacence fait gagner mémoire et temps dans les parcours. En Python, la liste d'adjacence se représente naturellement par un dictionnaire qui associe à chaque sommet une liste de couples (voisin, poids) ; la matrice sera une liste de listes. Dans tout ce cours, nous écrirons les listes d'adjacence dans un ordre fixé une fois pour toutes : l'ordre des voisins influence l'ordre des visites dans les parcours, et fixer une convention rend les résultats reproductibles.

L'essentiel
Matrice d'adjacence : n × n cases, test de voisinage immédiat, mais mémoire en n^2. Listes d'adjacence : mémoire proportionnelle aux arêtes, idéales pour les graphes creux et pour énumérer les voisins. La matrice d'un graphe non orienté est symétrique ; la somme de sa ligne i donne le degré du sommet i.

Exemple — Matrice et listes d'adjacence du graphe des quartiers

```
REPRESENTER LE GRAPHE DES QUARTIERS

1) Matrice d'adjacence (1 = route directe, 0 = aucune) :
ACI HIP MED BAD LAF MAG
ACI [ 0 1 1 1 0 0 ]
HIP [ 1 0 1 0 1 0 ]
MED [ 1 1 0 0 1 1 ]
BAD [ 1 0 0 0 1 1 ]
LAF [ 0 1 1 1 0 1 ]
MAG [ 0 0 1 1 1 0 ]
La matrice est symetrique : le graphe est non oriente.
La somme de la ligne MED vaut 4 : c'est deg(MED).

2) Listes d'adjacence (voisin : poids en km) :
ACI : HIP(2), MED(4), BAD(3)
HIP : ACI(2), MED(2), LAF(6)
MED : ACI(4), HIP(2), LAF(3), MAG(6)
BAD : ACI(3), MAG(2), LAF(7)
LAF : HIP(6), MED(3), BAD(7), MAG(4)
MAG : MED(6), BAD(2), LAF(4)

La matrice occupe 6 x 6 = 36 cases pour seulement
10 aretes : les listes sont plus economes ici.
```

À retenir

- Matrice : case (i, j) = 1 (ou le poids) si i et j sont voisins ; diagonale à 0 sans boucle ; symétrie = non orienté.
- Listes d'adjacence : mémoire en sommets + arêtes ; représentation naturelle des graphes creux, et celle des algorithmes du cours.
- Le choix dépend de la densité du graphe et des opérations fréquentes : tester un voisinage (matrice) ou énumérer les voisins (listes).

Exercice

On considère le graphe non orienté de sommets W, X, Y, Z et d'arêtes : W-X, W-Y, X-Z, Y-Z, X-Y. 1) Écris les listes d'adjacence (voisins dans l'ordre où les arêtes sont données, de gauche à droite). 2) Écris la matrice d'adjacence dans l'ordre W, X, Y, Z. 3) Donne le degré de chaque sommet et vérifie le lemme des poignées de main.

✔ Corrigé

1) Listes d'adjacence (on lit les arêtes dans l'ordre donné) : W : X, Y — X : W, Z, Y — Y : W, Z, X — Z : X, Y. (Chaque arête apparaît deux fois dans l'ensemble des listes, une fois chez chacune de ses extrémités : c'est normal en non orienté.) 2) Matrice (lignes et colonnes dans l'ordre W, X, Y, Z) : ligne W : 0 1 1 0 ; ligne X : 1 0 1 1 ; ligne Y : 1 1 0 1 ; ligne Z : 0 1 1 0. Elle est symétrique, comme attendu. 3) Degrés : deg(W) = 2, deg(X) = 3, deg(Y) = 3, deg(Z) = 2. Somme = 10 = 2 × 5 arêtes : le lemme est vérifié. On peut aussi lire chaque degré comme la somme de la ligne correspondante de la matrice.

Leçon 3 — Parcours en largeur et parcours en profondeur

Parcourir un graphe, c'est visiter tous ses sommets de façon systématique, sans en oublier ni en visiter deux fois. Le parcours en largeur (BFS, pour Breadth-First Search) explore le graphe par couches successives : on visite d'abord le sommet de départ, puis tous ses voisins, puis les voisins des voisins, et ainsi de suite. Pour y parvenir, on utilise une file — premier entré, premier sorti : on enfile les sommets au moment où on les découvre, et on les défile pour les explorer. Un point essentiel : un sommet est marqué comme visité dès qu'il entre dans la file, pas quand il en sort, sinon il risquerait d'être enfilé plusieurs fois. Propriété remarquable du BFS sur un graphe non pondéré : la première fois qu'un sommet est atteint, on l'a atteint par un plus court chemin en nombre d'arêtes. Le BFS calcule donc gratuitement les distances (en arêtes) depuis le départ.

Le parcours en profondeur (DFS, pour Depth-First Search) adopte la stratégie inverse : on suit un chemin aussi loin que possible, comme dans un labyrinthe, en gardant un fil d'Ariane. Arrivé à un sommet dont tous les voisins sont déjà visités, on revient en arrière jusqu'au dernier sommet qui offre encore une issue inexplorée. Cette mécanique de retour arrière correspond exactement au comportement d'une pile — dernier entré, premier sorti — ou, ce qui revient au même, à des appels récursifs : explorer un sommet, c'est le marquer puis explorer récursivement chacun de ses voisins non visités. Le DFS ne donne pas les plus courts chemins, mais il est très économe en mémoire et il excelle à détecter des structures : composantes connexes, cycles, ordre de traitement dans les graphes de dépendances.

Les deux parcours partagent le même squelette : marquer les sommets visités, sinon les cycles feraient boucler l'exploration indéfiniment. Leur ordre de visite dépend de l'ordre dans lequel on examine les voisins : deux parcours corrects du même graphe peuvent donc différer, et dans ce cours nous examinons toujours les voisins dans l'ordre des listes d'adjacence de la leçon 2. Complexité : sur un graphe représenté par listes d'adjacence, chaque sommet est traité une fois et chaque arête examinée au

plus deux fois, soit un temps proportionnel à (sommets + arêtes) pour les deux parcours. La différence est uniquement dans l'ordre — et dans la structure auxiliaire : file pour BFS, pile pour DFS.

L'essentiel
BFS = exploration par couches avec une file ; il donne les plus courts chemins en nombre d'arêtes dans un graphe non pondéré. DFS = exploration à fond d'un chemin avec une pile (ou la récursion), puis retour arrière. Dans les deux cas, on marque chaque sommet dès sa découverte pour ne jamais le traiter deux fois.

Exemple — BFS et DFS depuis ACI sur le graphe des quartiers

```
PARCOURS DEPUIS ACI (voisins dans l'ordre des listes)

PARCOURS EN LARGEUR (file : premier entre, premier sorti)
On retire ACI -> on enfile HIP, MED, BAD
file : HIP, MED, BAD
On retire HIP -> on enfile LAF
file : MED, BAD, LAF
On retire MED -> on enfile MAG
file : BAD, LAF, MAG
On retire BAD -> rien de nouveau ; file : LAF, MAG
On retire LAF -> rien de nouveau ; file : MAG
On retire MAG -> file vide, parcours termine.
Ordre de visite : ACI, HIP, MED, BAD, LAF, MAG
Distances en nombre d'aretes depuis ACI :
HIP = 1, MED = 1, BAD = 1, LAF = 2, MAG = 2

PARCOURS EN PROFONDEUR (on suit un chemin jusqu'au bout)
ACI -> HIP -> MED -> LAF -> BAD -> MAG
(depuis LAF, HIP et MED sont deja visites : on poursuit
vers BAD, puis vers MAG ; ensuite tout est visite,
on revient en arriere et le parcours se termine.)
Ordre de visite : ACI, HIP, MED, LAF, BAD, MAG
```

À retenir

- BFS = file (FIFO) ; DFS = pile (LIFO) ou récursion. Le marquage se fait à la découverte du sommet, jamais à sa sortie de la structure.
- BFS donne les distances minimales en nombre d'arêtes (graphe non pondéré) ; DFS ne les donne pas, mais consomme peu de mémoire.
- L'ordre de visite dépend de l'ordre d'examen des voisins : toujours annoncer sa convention avant de donner un parcours.

Exercice

On reprend le graphe de l'exercice de la leçon 2 (sommets W, X, Y, Z ; listes : W : X, Y — X : W, Z, Y — Y : W, Z, X — Z : X, Y). 1) Donne l'ordre de visite du parcours en largeur depuis W, en précisant l'évolution de la file. 2) Donne l'ordre de visite du parcours en profondeur depuis W. 3) Pourquoi les deux ordres diffèrent-ils alors qu'ils visitent les mêmes sommets ?

✔ Corrigé
1) BFS depuis W : on enfile W. On retire W, on enfile X et Y (file : X, Y). On retire X : ses voisins sont W (visité), Z (nouveau : on l'enfile) et Y (déjà dans la file) — file : Y, Z. On retire Y : aucun nouveau voisin (W, Z et X sont déjà connus). On retire Z : terminé. Ordre de visite : W, X, Y, Z — Z n'est découvert qu'au niveau 2, après tous les voisins directs de W. 2) DFS depuis W : on explore W, puis son premier voisin X ; depuis X, le premier voisin non visité dans la liste (X : W, Z, Y) est Z ; depuis Z, on poursuit vers Y, seul voisin non visité. Ordre de visite : W, X, Z, Y. 3) Les deux ordres diffèrent parce que les stratégies diffèrent : BFS traite d'abord tous les voisins du départ (X et Y) avant de s'éloigner vers Z, tandis que DFS s'enfonce immédiatement le long d'un chemin (W, X, Z) et ne revient à Y qu'ensuite. Les deux parcours visitent les mêmes sommets — le graphe est connexe — seul l'ordre change, et cet ordre dépend de la convention d'examen des voisins.

Leçon 4 — Chemins les plus courts : l'algorithme de Dijkstra

Dès que les arêtes ont des poids différents, le parcours en largeur ne suffit plus : un chemin de 3 arêtes peut être plus long, en kilomètres, qu'un chemin de 5 arêtes. Le problème du plus court chemin consiste, à partir d'un sommet source, à trouver pour chaque sommet le chemin de poids total minimal qui y mène. C'est le problème que résout ton GPS quand il choisit un itinéraire, et que résolvent les routeurs d'Internet pour acheminer les données. L'algorithme de Dijkstra, publié par l'informaticien néerlandais Edsger Dijkstra en 1959, est la solution classique de ce problème pour les graphes à poids positifs.

Le principe est glouton : on entretient pour chaque sommet une distance provisoire — la meilleure distance connue depuis la source — initialement 0 pour la source et infinie pour tous les autres. À chaque étape, on choisit le sommet non encore traité dont la distance provisoire est la plus petite, et on déclare cette distance définitive. Pourquoi est-elle définitive ? Parce que tout autre chemin vers ce sommet devrait passer par un sommet non traité, dont la distance provisoire est déjà supérieure ou égale, et que les poids étant positifs, la suite du chemin ne ferait qu'ajouter du poids : impossible donc de faire mieux. On met ensuite à jour les voisins du sommet choisi — c'est la relaxation : si passer par le sommet choisi améliore la distance provisoire d'un voisin, on la remplace et on note le sommet choisi comme prédécesseur de ce voisin.

Lorsque tous les sommets ont été traités, les distances provisoires sont les distances minimales cherchées, et les prédécesseurs permettent de reconstruire chaque plus court chemin en remontant de la destination vers la source. La condition des poids positifs (ou nuls) est indispensable : avec un poids négatif, un chemin apparemment plus long pourrait devenir meilleur après coup, et le choix « définitif » de Dijkstra deviendrait faux — un contre-exemple figure dans les exercices supplémentaires, et l'algorithme de Bellman-Ford prend alors le relais. Remarque enfin que si tous les poids valent 1, Dijkstra se comporte comme un parcours en largeur : le BFS est en quelque sorte son cas particulier. Dans sa version simple, à chaque étape on cherche le minimum parmi les sommets non traités, ce qui donne un coût en n^2 pour n

sommets ; une file de priorité (un tas) l'améliore nettement sur les grands graphes.

L'essentiel
Dijkstra : distances provisoires, on fige à chaque étape le sommet non traité le plus proche de la source, puis on relaxe ses arêtes. Condition absolue : poids positifs ou nuls. Les prédécesseurs permettent de reconstruire les plus courts chemins, pas seulement leurs longueurs.

Exemple — Dijkstra depuis ACI, étape par étape

```
ETAPES DE DIJKSTRA DEPUIS ACI (distances en km)

Choisi ACI HIP MED BAD LAF MAG
------ --- --- --- --- --- ---
(depart) 0 inf inf inf inf inf
ACI 0 2 4 3 inf inf
HIP 0 2 4 3 8 inf
BAD 0 2 4 3 8 5
MED 0 2 4 3 7 5
MAG 0 2 4 3 7 5
LAF 0 2 4 3 7 5

Lecture d'une etape, par exemple apres ACI :
HIP = 0 + 2 = 2 ; MED = 0 + 4 = 4 ; BAD = 0 + 3 = 3.
Apres HIP (choisi car 2 est le minimum non traite) :
LAF = 2 + 6 = 8 ; MED reste 4 (2 + 2 = 4, pas mieux).
Apres MED : LAF devient 4 + 3 = 7 (mieux que 8).

Resultats finaux : HIP 2, BAD 3, MED 4, MAG 5, LAF 7.
Plus court chemin ACI -> LAF : ACI - MED - LAF (4 + 3 = 7).
Remarque : ACI - HIP - MED - LAF vaut aussi 7 km (2+2+3) :
deux chemins optimaux peuvent coexister.
```

À retenir

- À chaque étape, le sommet non traité de plus petite distance provisoire devient définitif : c'est le cœur glouton de Dijkstra, valable uniquement sans poids négatif.
- La relaxation : dist(voisin) = min(dist(voisin), dist(sommet) + poids de l'arête). Noter le prédécesseur à chaque amélioration.
- Dijkstra donne les distances depuis la source vers TOUS les sommets, et les chemins complets par remontée des prédécesseurs.

Exercice

Un graphe pondéré a pour sommets S (départ), A, B, C et T, et pour arêtes : S-A : 1, S-B : 4, A-B : 2, A-C : 5, B-C : 1, C-T : 3, B-T : 6. Applique l'algorithme de Dijkstra depuis S en donnant le tableau des distances après chaque sommet choisi. Quel est le plus court chemin de S à T, et sa longueur ? Pourquoi le chemin direct S-B n'est-il pas le meilleur début de trajet ?

✔ Corrigé
Initialisation : dist(S) = 0, les autres à l'infini. On choisit S : A = 1, B = 4. On choisit A (distance 1, la plus petite) : B devient min(4, 1 + 2) = 3 ; C = 1 + 5 = 6. On choisit B (3) : C devient min(6, 3 + 1) = 4 ; T = 3 + 6 = 9. On choisit C (4) : T devient min(9, 4 + 3) = 7. On choisit T (7) : terminé. Distances finales : S = 0, A = 1, B = 3, C = 4, T = 7. En remontant les prédécesseurs depuis T (T ← C ← B ← A ← S), le plus court chemin est S - A - B - C - T, de longueur 1 + 2 + 1 + 3 = 7. Le chemin direct S-B coûte 4, alors que S-A-B ne coûte que 3 : l'arête directe la plus courte au départ n'est donc pas toujours le bon choix, et c'est exactement pour cela qu'il faut un algorithme plutôt qu'une intuition.

Leçon 5 — Arbres couvrants minimaux : Kruskal et Prim

Supposons que l'on veuille raccorder tous les quartiers de Bamako à la fibre optique : il faut que le réseau atteigne chaque quartier, sans boucle inutile (une boucle coûte du câble sans rien apporter), et au moindre coût total. Un arbre est un graphe connexe et sans cycle ; on démontre qu'un arbre à n sommets possède exactement n - 1 arêtes. Un arbre couvrant d'un graphe connexe est un arbre qui utilise tous les sommets du graphe et une partie de ses arêtes ; l'arbre couvrant minimal (ou de poids minimal) est celui dont la somme des poids est la plus petite possible. Attention à ne pas confondre avec les plus courts chemins : l'arbre couvrant minimal minimise un coût global de construction, pas les distances individuelles — un quartier peut y être atteint par un détour, si cela fait économiser du câble au total.

L'algorithme de Kruskal (1956) procède par appétit global : on trie toutes les arêtes par poids croissant, puis on les examine dans cet ordre en ajoutant chaque arête si elle relie deux composantes encore séparées, et en la rejetant si elle fermerait un cycle avec les arêtes déjà choisies. On entretient donc une forêt — un ensemble d'arbres partiels — qui fusionne peu à peu ; l'algorithme s'arrête quand il ne reste qu'un seul arbre, avec n - 1 arêtes. Pour savoir rapidement si une arête crée un cycle, on maintient pour chaque sommet l'étiquette de sa composante : si les deux extrémités ont des étiquettes différentes, on ajoute l'arête et on fusionne les étiquettes (c'est l'idée de la structure union-find). L'intuition de correction : à chaque choix, l'arête la moins chère reliant deux composantes figure forcément dans un arbre couvrant minimal.

L'algorithme de Prim (1957) adopte le point de vue opposé : un seul arbre qui grandit. On part d'un sommet quelconque ; à chaque étape, parmi toutes les arêtes qui relient un sommet de l'arbre à un sommet extérieur, on choisit la moins chère et on fait entrer son extrémité extérieure dans l'arbre. On répète jusqu'à couvrir tous les sommets. Kruskal compare toutes les arêtes du graphe, Prim ne regarde que la frontière de son arbre : sur un graphe dense, Prim est souvent plus simple à mener ; sur un graphe creux, le tri de Kruskal est très efficace. Les deux algorithmes donnent le même poids total minimal — en cas d'égalités de poids, les arbres obtenus peuvent différer, mais jamais leur poids. Retiens la philosophie commune : ce sont deux algorithmes gloutons, qui ne reviennent jamais sur un choix, et dont on peut prouver qu'ils sont

optimaux.

L'essentiel
Un arbre couvrant relie tous les sommets avec n - 1 arêtes et sans cycle ; le minimal minimise la somme des poids. Kruskal : trier les arêtes, ajouter sans créer de cycle. Prim : faire grandir un arbre par l'arête de frontière la moins chère. Même poids final garanti.

Exemple — Kruskal et Prim sur le graphe des quartiers

```
ARBRE COUVRANT MINIMAL DU GRAPHE DES QUARTIERS

ALGORITHME DE KRUSKAL (aretes triees par poids croissant) :
ACI-HIP (2) : AJOUTEE HIP-MED (2) : AJOUTEE
BAD-MAG (2) : AJOUTEE ACI-BAD (3) : AJOUTEE
MED-LAF (3) : AJOUTEE -> 5 aretes = 6 - 1 : STOP
Les aretes suivantes creeraient un cycle, par exemple
ACI-MED (4) : ACI et MED sont deja relies par le
chemin ACI-HIP-MED ; elle est donc REJETEE.
Total : 2 + 2 + 2 + 3 + 3 = 12 km de fibre a poser.

ALGORITHME DE PRIM (on part de ACI, l'arbre grandit) :
+ ACI-HIP (2) : plus petite arete sortant de {ACI}
+ HIP-MED (2) : plus petite arete sortant de {ACI, HIP}
+ ACI-BAD (3) : BAD rejoint l'arbre {ACI, HIP, MED}
+ BAD-MAG (2) : MAG rejoint l'arbre
+ MED-LAF (3) : LAF rejoint, arbre complet.
Total : 12 km. Meme poids que Kruskal : normal,
le poids minimal est unique meme si, en cas d'egalites
de poids, l'arbre obtenu peut varier.
```

À retenir

- Arbre couvrant : tous les sommets, n - 1 arêtes, aucun cycle. Minimal = somme des poids la plus faible possible.
- Kruskal rejette toute arête dont les extrémités sont déjà dans la même composante ; Prim ne choisit que des arêtes traversant la frontière de son arbre.
- Ne pas confondre : l'arbre couvrant minimal optimise un coût total de construction, Dijkstra optimise les trajets depuis une source.

Exercice

On veut relier cinq villages A, B, C, D, E par des pistes, avec les coûts suivants : A-B : 1, B-C : 2, A-C : 4, C-D : 3, B-D : 5, D-E : 2, C-E : 6. Applique l'algorithme de Kruskal en indiquant, pour chaque arête examinée, si elle est ajoutée ou rejetée et pourquoi. Quel est le coût total du réseau minimal, et combien d'arêtes contient-il ?

✔ Corrigé
On trie les arêtes par coût croissant : A-B (1), B-C (2), D-E (2), C-D (3), A-C (4), B-D (5), C-E (6). A-B (1) : ajoutée, composante {A, B}. B-C (2) : ajoutée, composante {A, B, C}. D-E (2) : ajoutée, composante {D, E}. C-D (3) : C et D sont dans des composantes différentes, ajoutée — les deux composantes fusionnent en {A, B, C, D, E}. Nous avons 4 arêtes, soit 5 - 1 : l'arbre est complet, on s'arrête sans examiner les arêtes restantes, qui seraient d'ailleurs toutes rejetées (A-C fermerait le cycle A-B-C, B-D et C-E relient des sommets déjà dans la même composante). Coût total : 1 + 2 + 2 + 3 = 8, pour 4 arêtes. Prim, en partant de A, choisirait A-B (1), B-C (2), C-D (3), D-E (2) : même coût total de 8, comme garanti.

Leçon 6 — Coloration de graphes et problèmes de planification

Colorier un graphe, c'est attribuer une couleur à chaque sommet de sorte que deux sommets voisins n'aient jamais la même couleur — c'est ce qu'on appelle une coloration propre. Le nombre chromatique d'un graphe, noté χ(G) (« khi »), est le plus petit nombre de couleurs qui permette une coloration propre. Certains obstacles imposent des bornes immédiates : si le graphe contient un triangle (trois sommets tous reliés entre eux), il faut au moins 3 couleurs, car ces trois sommets sont voisins deux à deux ; plus généralement, une clique de taille k impose au moins k couleurs. À l'inverse, un graphe sans aucun cycle impair — en particulier un arbre ou un cycle pair — se colorie avec 2 couleurs seulement : on parle de graphe biparti. Entre la borne inférieure donnée par les cliques et ce que produisent les algorithmes, toute la difficulté du problème se joue.

L'algorithme le plus simple est l'algorithme glouton : on prend les sommets dans un certain ordre et on donne à chacun la plus petite couleur encore libre parmi ses voisins déjà coloriés. Résultat important : l'ordre choisi change tout. Un mauvais ordre peut gaspiller des couleurs, un bon ordre — par exemple les sommets triés par degré décroissant (heuristique de Welsh et Powell) — donne souvent l'optimum, mais pas toujours : le glouton ne garantit pas le nombre chromatique. Ce n'est pas un défaut de paresse : calculer le nombre chromatique exact d'un grand graphe quelconque est un problème réputé très difficile, pour lequel aucun algorithme rapide général n'est connu. En pratique, on se contente donc d'heuristiques efficaces, et on sait vérifier qu'une coloration est propre en examinant chaque arête une seule fois.

Les applications de la coloration sont partout où des événements incompatibles se disputent des ressources. Planifier des examens : les sommets sont les examens, une arête relie deux examens ayant des étudiants en commun, et les couleurs sont les créneaux horaires — le nombre de couleurs est le nombre de créneaux nécessaires. Même modèle pour les emplois du temps scolaires, l'attribution de fréquences radio à des émetteurs voisins (deux émetteurs proches ne doivent pas émettre sur la même fréquence), l'affectation de registres dans un compilateur, ou encore le sudoku, qui est exactement la coloration d'un graphe de 81 sommets. À chaque fois, la démarche est la même : modéliser les conflits par des arêtes, puis chercher une coloration avec peu de couleurs.

L'essentiel
Une coloration propre attribue à chaque sommet une couleur différente de celles de ses voisins. Le nombre chromatique χ(G) est le minimum de couleurs nécessaires : un triangle impose χ ≥ 3, un graphe biparti a χ = 2. Le glouton (surtout par degrés décroissants) approche souvent l'optimum sans le garantir.

Exemple — Colorier les examens de fin de semestre à l'IAM

```
COLORIER LES EXAMENS DE FIN DE SEMESTRE A L'IAM

Deux examens sont relies s'ils ont des etudiants en commun
(ils ne peuvent donc pas avoir lieu au meme creneau) :
Maths (M) : I, P
Informatique (I): M, P, A
Physique (P) : M, I, E
Anglais (A) : I, E, H
Economie (E) : P, A, H
Histoire (H) : A, E
M, I et P forment un triangle : il faut au moins
3 creneaux, quoi qu'on fasse.

COLORATION GLOUTONNE (degres decroissants : I, P, A, E, M, H) :
I -> creneau 1 (aucun voisin colorie)
P -> creneau 2 (I est au creneau 1)
A -> creneau 2 (I est au 1 ; P n'est pas son voisin)
E -> creneau 1 (ses voisins P et A sont au 2)
M -> creneau 3 (voisins I au 1 et P au 2)
H -> creneau 3 (voisins A au 2 et E au 1)

PLANNING EN 3 CRENEAUX :
Creneau 1 : Informatique, Economie
Creneau 2 : Physique, Anglais
Creneau 3 : Maths, Histoire
3 creneaux = le minimum possible (a cause du triangle) :
la coloration obtenue est donc optimale.
```

À retenir

- Coloration propre : jamais la même couleur aux deux extrémités d'une arête. Le nombre chromatique χ(G) est le minimum de couleurs.
- Bornes utiles : une clique de taille k impose χ ≥ k ; un graphe biparti (sans cycle impair) a χ = 2 ; le glouton donne au plus Δ + 1 couleurs (Δ = degré maximum).
- Modéliser un planning = sommets (événements), arêtes (conflits), couleurs (créneaux ou ressources). Le glouton par degrés décroissants est l'heuristique de base.

Exercice

Cinq cours C1, C2, C3, C4, C5 doivent être placés dans des créneaux horaires. Les conflits (étudiants inscrits aux deux cours) sont : C1-C2, C1-C3, C2-C3, C3-C4, C4-C5, C2-C5. 1) Colorie ce graphe avec l'algorithme glouton en prenant les cours dans l'ordre C1, C2, C3, C4, C5. 2) Donne le planning correspondant. 3) Aurait-on pu se contenter

de 2 créneaux ? Justifie.

✔ Corrigé
1) C1 reçoit le créneau 1. C2, voisin de C1, reçoit le créneau 2. C3 est voisin de C1 (créneau 1) et de C2 (créneau 2) : il reçoit le créneau 3. C4 n'est voisin que de C3 (créneau 3) parmi les cours déjà placés : il reçoit le créneau 1. C5 est voisin de C4 (créneau 1) et de C2 (créneau 2) : il reçoit le créneau 3. 2) Planning : créneau 1 : C1 et C4 (aucun conflit entre eux) ; créneau 2 : C2 seul ; créneau 3 : C3 et C5. Vérification rapide : chaque paire en conflit est bien séparée. 3) Non : C1, C2 et C3 sont deux à deux en conflit (c'est un triangle), donc ils exigent trois créneaux distincts. Le nombre chromatique vaut exactement 3, et la coloration gloutonne est ici optimale.

Leçon 7 — Applications : réseaux, transport et réseaux sociaux

Dans le transport et la navigation, les graphes sont l'outil de base : les sommets sont des lieux (carrefours, gares, aéroports), les arêtes des routes ou des liaisons, et les poids des distances ou des temps de trajet. Les GPS et applications de cartographie y exécutent des variantes de Dijkstra pour proposer l'itinéraire le plus rapide, en actualisant les poids avec le trafic. Internet lui-même fonctionne ainsi : le protocole de routage OSPF, utilisé dans les grands réseaux, fait calculer à chaque routeur les plus courts chemins vers toutes les destinations par l'algorithme de Dijkstra. Le parcours en largeur intervient dès qu'on compte en nombre d'étapes : nombre minimal de correspondances entre deux gares, ou nombre de sauts entre deux machines.

Pour construire des réseaux physiques — fibre optique, électricité, eau, routes — l'arbre couvrant minimal donne le plan de raccordement le moins cher qui relie tout le monde. Mais l'ingénieur doit connaître sa limite : un arbre n'a aucune redondance, la coupure d'une seule liaison y déconnecte une partie du réseau. En pratique, on part donc de l'arbre couvrant minimal, puis on ajoute quelques liaisons supplémentaires aux endroits stratégiques pour créer des cycles de secours, en acceptant un surcoût maîtrisé. Le même raisonnement vaut pour les réseaux de capteurs ou de distribution : l'arbre minimal est le squelette économique, auquel on ajoute de la robustesse là où une panne serait inacceptable.

Les réseaux sociaux se modélisent de la même façon : sommets = personnes ou comptes, arêtes = relations (amitié, suivi, messages). Le degré mesure la connectivité directe d'un membre ; un parcours en largeur donne les « degrés de séparation » entre deux personnes — la fameuse idée que quelques intermédiaires suffisent à relier deux inconnus ; les suggestions d'amis exploitent les voisins communs, c'est-à-dire les triangles du graphe. On retrouve encore des graphes dans les dépendances entre tâches d'un projet, les pages et liens du web, les états d'un jeu ou d'un protocole. Le réflexe professionnel à acquérir est toujours le même : identifier ce que sont les sommets, les arêtes et les poids, puis reconnaître le problème — parcours, plus court chemin, arbre couvrant ou coloration — pour appliquer l'algorithme adapté.

L'essentiel
Transport et GPS : plus courts chemins pondérés (Dijkstra). Construction de réseaux au moindre coût : arbre couvrant minimal, complété par des liaisons de secours. Réseaux sociaux : degré = connectivité, BFS = degrés de séparation, triangles = suggestions d'amis. Tout commence par une bonne modélisation.

Exemple — Trois problèmes, trois algorithmes du cours

```
TROIS PROBLEMES, TROIS ALGORITHMES DU COURS

1) Itineraire d'un livreur a Bamako (GPS, livraison...)
Probleme : plus court chemin entre deux quartiers.
Outil : Dijkstra. ACI -> MAG = 5 km par ACI-BAD-MAG,
alors que l'itineraire par MED vaudrait 4 + 6 = 10 km.

2) Raccorder les quartiers a la fibre optique
Probleme : relier les 6 quartiers en depensant le
moins de cable possible, sans boucle inutile.
Outil : arbre couvrant minimal (Kruskal ou Prim) :
12 km de fibre suffisent, contre 39 km si l'on
posait du cable le long des 10 routes existantes.

3) Detecter les personnes influentes d'un reseau social
Probleme : qui est le plus connecte ?
Outil : le degre. Dans notre graphe, MED et LAF
(degre 4) sont les carrefours les plus importants ;
dans un reseau social, ce sont les comptes relais
par lesquels l'information circule le plus vite.
```

À retenir

- GPS et routage Internet (OSPF) : des plus courts chemins de Dijkstra, recalculés en permanence avec des poids actualisés.
- Un arbre couvrant minimal est le squelette le moins cher d'un réseau, mais sans aucune redondance : les vrais réseaux ajoutent des cycles de secours.
- Face à un problème nouveau : nommer sommets, arêtes et poids, puis identifier le type de question (parcours, chemin, arbre, coloration).

Exercice

On revient au graphe des quartiers de Bamako du fil rouge. 1) Une ONG veut installer son centre de distribution dans le quartier le mieux connecté aux autres par des routes directes. Quels quartiers sont candidats, et pourquoi ? 2) Un livreur part d'ACI 2000 et doit livrer à Magnambougou : quel est l'itinéraire le plus court et sa longueur ? (Utilise le résultat de la leçon 4.) 3) Explique pourquoi le critère du degré, seul, ne suffit pas toujours à choisir le meilleur emplacement.

✔ Corrigé
1) Le degré mesure le nombre de routes directes : deg(MED) = deg(LAF) = 4, contre 3 pour tous les autres quartiers. Médina Coura et Lafiabougou sont donc les mieux connectés et les candidats naturels pour le centre de distribution. 2) D'après Dijkstra depuis ACI (leçon 4), la distance minimale d'ACI à MAG est 5 km, par le chemin ACI - BAD - MAG (3 + 2). L'itinéraire par Médina Coura vaudrait 4 + 6 = 10 km, le double : il faut donc passer par Badalabougou. 3) Le degré ne tient compte que du nombre de liaisons, pas de leurs longueurs ni de la position du quartier par rapport à l'ensemble : LAF a le degré maximal (4) mais se trouve à 7 km d'ACI, alors que BAD, de degré 3 seulement, n'est qu'à 3 km. Pour un choix réel, on combinerait le degré avec les distances de Dijkstra depuis chaque candidat (par exemple la somme des distances vers tous les autres quartiers).

Travaux dirigés (TD)

TD 1 — TD 1 — Dijkstra : les tournées d'un livreur

Une société de livraison a son dépôt au point A et dessert les clients B, C, D, E et F. Les temps de trajet (en minutes) entre points voisins sont : A-B : 4, A-C : 2, C-B : 1, C-D : 5, B-D : 2, B-E : 6, D-E : 3, D-F : 4, E-F : 2, C-E : 7. 1) Applique l'algorithme de Dijkstra depuis A et donne la distance minimale du dépôt vers chaque client, avec le tableau des étapes. 2) Donne l'itinéraire optimal du dépôt jusqu'à F, puis jusqu'à E. 3) Un chauffeur propose l'itinéraire A - B - D - F : calcule son temps et compare-le à l'optimum.

✔ Corrigé du TD 1
1) Initialisation : A = 0, les autres à l'infini. On choisit A : B = 4, C = 2. On choisit C (2) : B devient min(4, 2 + 1) = 3 ; D = 2 + 5 = 7 ; E = 2 + 7 = 9. On choisit B (3) : D devient min(7, 3 + 2) = 5 ; E reste 9 (3 + 6 = 9, pas d'amélioration). On choisit D (5) : E devient min(9, 5 + 3) = 8 ; F = 5 + 4 = 9. On choisit E (8) : F reste 9 (8 + 2 = 10). On choisit F (9). Distances finales : A = 0, C = 2, B = 3, D = 5, E = 8, F = 9. 2) Prédécesseurs : F ← D ← B ← C ← A, donc l'itinéraire optimal vers F est A - C - B - D - F, en 2 + 1 + 2 + 4 = 9 minutes. Pour E : A - C - B - D - E, en 8 minutes. 3) L'itinéraire proposé coûte 4 + 2 + 4 = 10 minutes : une minute de plus que l'optimum, parce que le détour par C au départ (2 minutes) fait gagner sur la suite du trajet. Moralité : l'optimum global ne se devine pas, il se calcule.

TD 2 — TD 2 — Arbre couvrant minimal : la fibre du campus

L'université veut relier six bâtiments par la fibre optique : Administration (A), Bibliothèque (B), Salles de cours (S), Laboratoire (L), Restaurant (R), Gymnase (G). Les coûts de pose, en dizaines de mètres de câble, sont : A-B : 3, A-S : 5, B-S : 2, B-L : 4, S-L : 3, S-R : 6, L-R : 2, L-G : 5, R-G : 3, B-R : 7. 1) Applique l'algorithme de Kruskal pour obtenir le plan de raccordement le moins cher, en justifiant chaque arête rejetée. 2) Applique l'algorithme de Prim en partant de l'Administration. 3) Compare les deux résultats : nombre d'arêtes, coût total en mètres de câble.

✔ Corrigé du TD 2
1) Kruskal — arêtes triées : B-S (2), L-R (2), A-B (3), S-L (3), R-G (3), B-L (4), A-S (5), L-G (5), S-R (6), B-R (7). B-S (2) : ajoutée ({B, S}). L-R (2) : ajoutée ({L, R}). A-B (3) : ajoutée ({A, B, S}). S-L (3) : S et L sont dans des composantes différentes, ajoutée — fusion en {A, B, S, L, R}. R-G (3) : ajoutée, G rejoint l'ensemble. On a 5 arêtes = 6 - 1 : stop. Les suivantes auraient été rejetées : B-L (4) fermerait un cycle dans {A, B, S, L} ; A-S (5), L-G (5), S-R (6) et B-R (7) relient des sommets déjà dans la même composante. Coût total : 2 + 2 + 3 + 3 + 3 = 13. 2) Prim depuis A : on ajoute A-B (3), l'arête la moins chère sortant de {A} ; puis B-S (2) ; puis S-L (3) ; puis L-R (2) ; puis R-G (3). Coût : 3 + 2 + 3 + 2 + 3 = 13. 3) Les deux algorithmes retiennent les mêmes 5 arêtes ici et donnent le même coût total minimal : 13 dizaines de mètres, soit 130 mètres de câble. C'est la propriété générale : l'arbre peut varier selon les égalités de poids, mais jamais le poids minimal.

Travaux pratiques (TP)

TP 1 — TP — Programmer BFS et Dijkstra sur le graphe des quartiers

En Python : 1) Représente le graphe des quartiers du fil rouge par un dictionnaire associant à chaque sommet la liste de ses couples (voisin, distance en km). 2) Écris une fonction de parcours en largeur qui renvoie, pour chaque quartier, sa distance en nombre d'arêtes depuis ACI 2000 (utilise collections.deque pour la file). 3) Écris l'algorithme de Dijkstra avec le module heapq (file de priorité) pour obtenir les distances en kilomètres depuis ACI 2000. 4) Vérifie par programme que la somme des degrés vaut 20 et que ton Dijkstra retrouve bien : HIP 2, BAD 3, MED 4, MAG 5, LAF 7.

✔ Attendus du TP 1
Représentation attendue : un dictionnaire du type {"ACI": [("HIP", 2), ("MED", 4), ("BAD", 3)], ...} reprenant les 10 arêtes, chacune dans les deux sens. Pour le BFS, on marque chaque sommet au moment de l'enfiler (pas à sa sortie de file) et on propage les distances en arêtes : le programme doit afficher ACI : 0, HIP : 1, MED : 1, BAD : 1, LAF : 2, MAG : 2. Pour Dijkstra, on stocke dans le tas des couples (distance, sommet) ; quand on dépile un couple dont la distance est supérieure à la distance déjà enregistrée, on l'ignore (c'est une entrée périmée) ; sinon on relaxe les voisins. Résultat attendu : ACI : 0, HIP : 2, MED : 4, BAD : 3, LAF : 7, MAG : 5 — ce sont exactement les valeurs calculées à la main dans la leçon 4. La somme des degrés se vérifie en additionnant les longueurs des listes d'adjacence : elle vaut 20, soit 2 × 10 arêtes. Points de vigilance : initialiser les distances à l'infini (float("inf")) sauf la source, et tester le code sur le graphe de l'exercice de la leçon 4 (S, A, B, C, T) dont on connaît la réponse : T doit être à 7.

Exercices corrigés supplémentaires

Exercice 1

Exercice 1 — Degrés dans un graphe orienté. Un graphe orienté a pour sommets P, Q, R, S et pour arcs : P→Q, P→S, Q→R, R→P, R→S, S→Q. Écris sa matrice d'adjacence (ligne

= sommet de départ, colonne = sommet d'arrivée, dans l'ordre P, Q, R, S), puis donne les degrés entrants et sortants de chaque sommet et vérifie les deux sommes.

✔ Corrigé
Matrice d'adjacence : ligne P : 0 1 0 1 ; ligne Q : 0 0 1 0 ; ligne R : 1 0 0 1 ; ligne S : 0 1 0 0. (Contrairement au cas non orienté, elle n'est pas symétrique : par exemple il existe un arc P→S mais pas d'arc S→P.) Degrés sortants (sommes des lignes) : P = 2, Q = 1, R = 2, S = 1, total 6. Degrés entrants (sommes des colonnes) : P = 1 (depuis R), Q = 2 (depuis P et S), R = 1 (depuis Q), S = 2 (depuis P et R), total 6. Les deux sommes valent le nombre d'arcs, 6 : chaque arc compte une fois comme départ et une fois comme arrivée.

Exercice 2

Exercice 2 — Un graphe qui se colorie avec 2 couleurs. On considère le cycle de 6 sommets 1-2-3-4-5-6-1 auquel on ajoute les deux cordes 1-4 et 2-5. Montre que ce graphe est coloriable avec 2 couleurs, en donnant explicitement la coloration, et explique pourquoi.

✔ Corrigé
Attribuons la couleur 1 aux sommets impairs {1, 3, 5} et la couleur 2 aux sommets pairs {2, 4, 6}. Vérifions toutes les arêtes : le cycle relie toujours un impair à un pair (1-2, 2-3, 3-4, 4-5, 5-6, 6-1) ; la corde 1-4 relie un impair à un pair ; la corde 2-5 également. Aucune arête n'a donc ses deux extrémités de même couleur : la coloration est propre avec 2 couleurs. L'explication générale : le graphe ne contient que des cycles de longueur paire (le cycle principal a 6 sommets, et les cordes créent des cycles de 4 sommets : 1-2-5-6-1 et 2-3-4-5-2). Un graphe sans cycle impair est biparti, donc coloriable avec 2 couleurs — on peut d'ailleurs obtenir cette coloration par un parcours en largeur, en alternant les couleurs à chaque niveau.

Exercice 3

Exercice 3 — Pourquoi Dijkstra interdit les poids négatifs. On considère le graphe orienté pondéré : A→B de poids 2, A→C de poids 3, C→B de poids −2. 1) Que calcule l'algorithme de Dijkstra depuis A ? 2) Quel est en réalité le plus court chemin de A à B ? 3) Conclus.

✔ Corrigé
1) Dijkstra initialise dist(A) = 0, puis choisit A et relaxe : dist(B) = 2, dist(C) = 3. Le sommet non traité le plus proche est B (2 < 3) : Dijkstra fige dist(B) = 2 comme définitive et traite ensuite C — mais depuis C on découvre un chemin A→C→B de longueur 3 + (−2) = 1, meilleur que 2, alors que B est déjà figé. L'algorithme rend donc 2, un résultat faux. 2) Le vrai plus court chemin est A→C→B, de longueur 1. 3) Conclusion : le raisonnement « le plus proche non traité est définitif » s'effondre dès qu'un poids négatif existe, car la fin d'un chemin peut le raccourcir au lieu de l'allonger. Dijkstra exige donc des poids positifs ou nuls ; avec des poids négatifs, on utilise l'algorithme de Bellman-Ford, plus lent mais correct.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans un graphe non orienté, la somme des degrés de tous les sommets est toujours égale…

   A) Au nombre de sommets du graphe

   B) Au double du nombre d'arêtes

   C) Au nombre d'arêtes du graphe

2. Une arête qui relie un sommet à lui-même s'appelle…

   A) Un cycle

   B) Un pont

   C) Une boucle

3. La matrice d'adjacence d'un graphe non orienté est toujours…

   A) Symétrique par rapport à la diagonale

   B) Composée uniquement de 0 et de 2

   C) Triangulaire supérieure

4. Le parcours en largeur (BFS) explore le graphe à l'aide…

   A) D'une pile

   B) D'un arbre couvrant

   C) D'une file

5. Dans un graphe non pondéré, le parcours en largeur depuis un sommet donne…

   A) Un arbre couvrant de poids minimal

   B) Les plus courts chemins en nombre d'arêtes

   C) Le degré de chaque sommet

6. L'algorithme de Dijkstra ne fonctionne correctement que si…

   A) Tous les poids sont positifs ou nuls

   B) Le graphe est orienté

   C) Le graphe contient moins de 100 sommets

7. Un arbre couvrant d'un graphe connexe à 8 sommets possède exactement…

   A) 8 arêtes

   B) Autant d'arêtes que le graphe d'origine

   C) 7 arêtes

8. L'algorithme de Kruskal commence par…

   A) Choisir un sommet de départ

   B) Trier les arêtes par poids croissant

   C) Colorier les sommets par degré

9. Si un graphe contient un triangle (trois sommets tous reliés entre eux), alors son nombre chromatique est au moins…

A) 3

B) 2

C) Cela dépend des poids des arêtes

10. Laquelle de ces situations se modélise naturellement par une coloration de graphe ?

A) Calculer l'itinéraire le plus rapide en voiture

B) Raccorder des bâtiments à la fibre au moindre coût

C) Planifier des examens sans conflit d'étudiants

Corrigés du QCM

1. Réponse B — Chaque arête a deux extrémités et contribue donc pour 2 à la somme des degrés : c'est le lemme des poignées de main.

2. Réponse C — Une boucle relie un sommet à lui-même et compte double dans son degré ; un graphe simple n'en contient pas.

3. Réponse A — Si le sommet i est voisin de j, alors j est voisin de i : la case (i, j) égale toujours la case (j, i).

4. Réponse C — La file (premier entré, premier sorti) fait visiter les sommets niveau par niveau, des plus proches aux plus éloignés.

5. Réponse B — BFS découvre chaque sommet par un chemin utilisant le moins d'arêtes possible ; c'est pourquoi la distance BFS compte en arêtes, pas en kilomètres.

6. Réponse A — Avec un poids négatif, un chemin pourrait s'améliorer après coup et le sommet déjà figé par Dijkstra aurait une distance fausse.

7. Réponse C — Tout arbre à n sommets possède exactement $n - 1$ arêtes : ici $8 - 1 = 7$.

8. Réponse B — Kruskal trie toutes les arêtes, puis les ajoute dans cet ordre en rejetant celles qui créeraient un cycle.

9. Réponse A — Les trois sommets du triangle sont voisins deux à deux : il leur faut trois couleurs distinctes. Les poids ne jouent aucun rôle en coloration.

10. Réponse C — Sommets = examens, arêtes = conflits (étudiants communs), couleurs = créneaux horaires : c'est exactement le modèle de la leçon 6.

Résumé — les notions importantes

• Un graphe G = (S, A) : sommets reliés par des arêtes (non orienté) ou des arcs (orienté) ; somme des degrés = 2 × nombre d'arêtes.

• Deux représentations : matrice d'adjacence (n × n, test de voisinage immédiat) et listes d'adjacence (économes pour les graphes creux).

• BFS explore par couches avec une file et donne les plus courts chemins en nombre d'arêtes ; DFS s'enfonce avec une pile ou la récursion.

• Dijkstra calcule les plus courts chemins pondérés depuis une source en figeant à chaque étape le sommet non traité le plus proche — poids positifs obligatoires.

• Kruskal (tri des arêtes, aucun cycle) et Prim (croissance d'un arbre) construisent un arbre couvrant minimal de même poids total, avec n − 1 arêtes.

• La coloration assigne des couleurs sans conflit entre voisins ; le nombre chromatique borne le nombre de créneaux d'un planning sans conflit.

• Savoir modéliser — sommets, arêtes, poids — puis choisir l'algorithme adapté : c'est la compétence centrale de la théorie des graphes.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Définitions : graphes orientés et non orientés
G = (S, A) : sommets + arêtes. Orienté = arcs fléchés (degré entrant / degré sortant) ; non orienté = arêtes utilisables dans les deux sens. ; Lemme des poignées de main : somme des degrés = 2 × nombre d'arêtes — un contrôle rapide pour vérifier un graphe dessiné ou saisi. ; Pondéré, connexe, chemin, cycle, boucle : un petit vocabulaire précis qui conditionne le choix de l'algorithme à appliquer.

Fiche 2 — Représentations : matrices et listes d'adjacence
Matrice : case (i, j) = 1 (ou le poids) si i et j sont voisins ; diagonale à 0 sans boucle ; symétrie = non orienté. ; Listes d'adjacence : mémoire en sommets + arêtes ; représentation naturelle des graphes creux, et celle des algorithmes du cours. ; Le choix dépend de la densité du graphe et des opérations fréquentes : tester un voisinage (matrice) ou énumérer les voisins (listes).

Fiche 3 — Parcours en largeur et parcours en profondeur
BFS = file (FIFO) ; DFS = pile (LIFO) ou récursion. Le marquage se fait à la découverte du sommet, jamais à sa sortie de la structure. ; BFS donne les distances minimales en nombre d'arêtes (graphe non pondéré) ; DFS ne les donne pas, mais consomme peu de mémoire. ; L'ordre de visite dépend de l'ordre d'examen des voisins : toujours annoncer sa convention avant de donner un parcours.

Fiche 4 — Chemins les plus courts : l'algorithme de Dijkstra
À chaque étape, le sommet non traité de plus petite distance provisoire devient définitif : c'est le cœur glouton de Dijkstra, valable uniquement sans poids négatif. ; La relaxation : dist(voisin) = min(dist(voisin), dist(sommet) + poids de l'arête). Noter le prédécesseur à chaque amélioration. ; Dijkstra donne les distances depuis la source vers TOUS les sommets, et les chemins complets par remontée des prédécesseurs.

Fiche 5 — Arbres couvrants minimaux : Kruskal et Prim
Arbre couvrant : tous les sommets, n - 1 arêtes, aucun cycle. Minimal = somme des poids la plus faible possible. ; Kruskal rejette toute arête dont les extrémités sont déjà dans la même composante ; Prim ne choisit que des arêtes traversant la frontière de son arbre. ; Ne pas confondre : l'arbre couvrant minimal optimise un coût total de construction, Dijkstra optimise les trajets depuis une source.

Fiche 6 — Coloration de graphes et problèmes de planification
Coloration propre : jamais la même couleur aux deux extrémités d'une arête. Le nombre chromatique $\chi(G)$ est le minimum de couleurs. ; Bornes utiles : une clique de taille k impose $\chi \geq k$; un graphe biparti (sans cycle impair) a $\chi = 2$; le glouton donne au plus $\Delta + 1$ couleurs (Δ = degré maximum). ; Modéliser un planning = sommets (événements), arêtes (conflits), couleurs (créneaux ou ressources). Le glouton par degrés décroissants est l'heuristique de base.

Fiche 7 — Applications : réseaux, transport et réseaux sociaux
GPS et routage Internet (OSPF) : des plus courts chemins de Dijkstra, recalculés en permanence avec des poids actualisés. ; Un arbre couvrant minimal est le squelette le moins cher d'un réseau, mais sans aucune redondance : les vrais réseaux ajoutent des cycles de secours. ; Face à un problème nouveau : nommer sommets, arêtes et poids, puis identifier le type de question (parcours, chemin, arbre, coloration).

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 3 — Les Fondamentaux de l'IA : Recherche Non Informée (BFS vs DFS) - Résoudre Puzzles et Labyrinthes

Chaîne : Coding with Iannaki · Langue : Français · Niveau : Débutant à intermédiaire
Une présentation claire des deux parcours fondamentaux, comparés pas à pas sur des puzzles et des labyrinthes. Idéal pour visualiser la différence entre la file du BFS et la pile du DFS avant de les coder en TP.

https://www.youtube.com/watch?v=nnSus1UD_zo

Leçon 4 — Dijkstra, le plus court chemin dans un graphe

Chaîne : PhysAlgo · Langue : Français · Niveau : Débutant
L'algorithme de Dijkstra expliqué de façon visuelle sur un graphe simple, avec le remplissage progressif du tableau des distances. Un bon complément à l'exécution manuelle de la leçon 4 sur les quartiers de Bamako.

https://www.youtube.com/watch?v=J8AntuTp1K4

Leçon 4 — Utiliser l'algorithme de Dijkstra - PostBac

Chaîne : Yvan Monka · Langue : Français · Niveau : Débutant à intermédiaire
Un exercice complet traité avec la méthode du tableau, dans le style méthodique des vidéos d'Yvan Monka. Parfait pour s'entraîner à dérouler Dijkstra sans sauter d'étape,

comme dans le TD 1.

https://www.youtube.com/watch?v=rHylCtXtdNs

Leçon 4 — Algorithme de Dijkstra : théorie des graphes

Chaîne : Méthode Maths · Langue : Français · Niveau : Intermédiaire

Une présentation plus théorique de Dijkstra dans le cadre de la théorie des graphes, avec le vocabulaire complet (chaînes, poids, ordre). Utile pour consolider les définitions des leçons 1 et 4.

https://www.youtube.com/watch?v=XGmpZJOGSPc

Leçon 5 — Algorithme de Kruskal pour construire un arbre couvrant de poids minimal

Chaîne : À la découverte des graphes · Langue : Français · Niveau : Intermédiaire

La construction d'un arbre couvrant minimal par l'algorithme de Kruskal, arête après arête, avec le rejet des arêtes qui fermeraient un cycle. La même chaîne propose aussi une vidéo sur l'algorithme de Prim pour comparer les deux approches.

https://www.youtube.com/watch?v=U_itW96NnQ0

Sources et références

- Programme officiel de Licence Informatique — Université IAM, Bamako (Mali).
- T. Cormen, C. Leiserson, R. Rivest, C. Stein — « Introduction à l'algorithmique », Dunod : chapitres sur les parcours de graphes, Dijkstra et les arbres couvrants minimaux.
- M. Gondran, M. Minoux — « Graphes et algorithmes », Eyrolles : référence francophone classique sur les graphes et leurs applications.
- Documentation officielle Python : modules heapq et collections (deque) pour implémenter Dijkstra et le parcours en largeur.
- Les vidéos pédagogiques citées dans la section « Vidéos associées » de ce document.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.